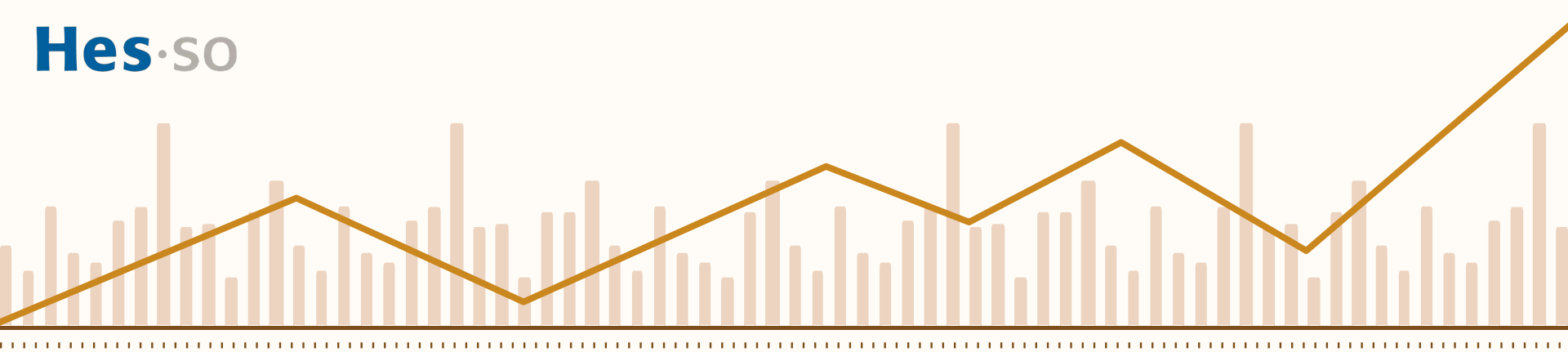


MLOPS PROJECT

DATA DRIFT OBSERVATION

IN A STOCK PREDICTION MODEL

CARDOSO RAFAEL, DA SILVA LUCA, FERRARA JUSTIN, MAKSIMOUS THOMAS

TSM-MachLeData
Bertil Chapuis
Beat Wolf

Project Overview

Goal: Predict 5-day stock returns from the past 30 days of price variations.



Dataset

20 tech stocks, 5 years of historical NASDAQ data



Challenges

High volatility (daily swings up to ±3%)
Concept drift & data drift
Need for reproducible MLOps pipeline

Concept Drift and Data Drift

1 Concept Drift

The relationship between X and Y transforms

- New macroeconomic effects
- New regulations

2 Data Drift – Feature Drift

Input variables change

- Market volatility increases
- Stock Volume change

3 Data Drift – Label Drift

The target changes

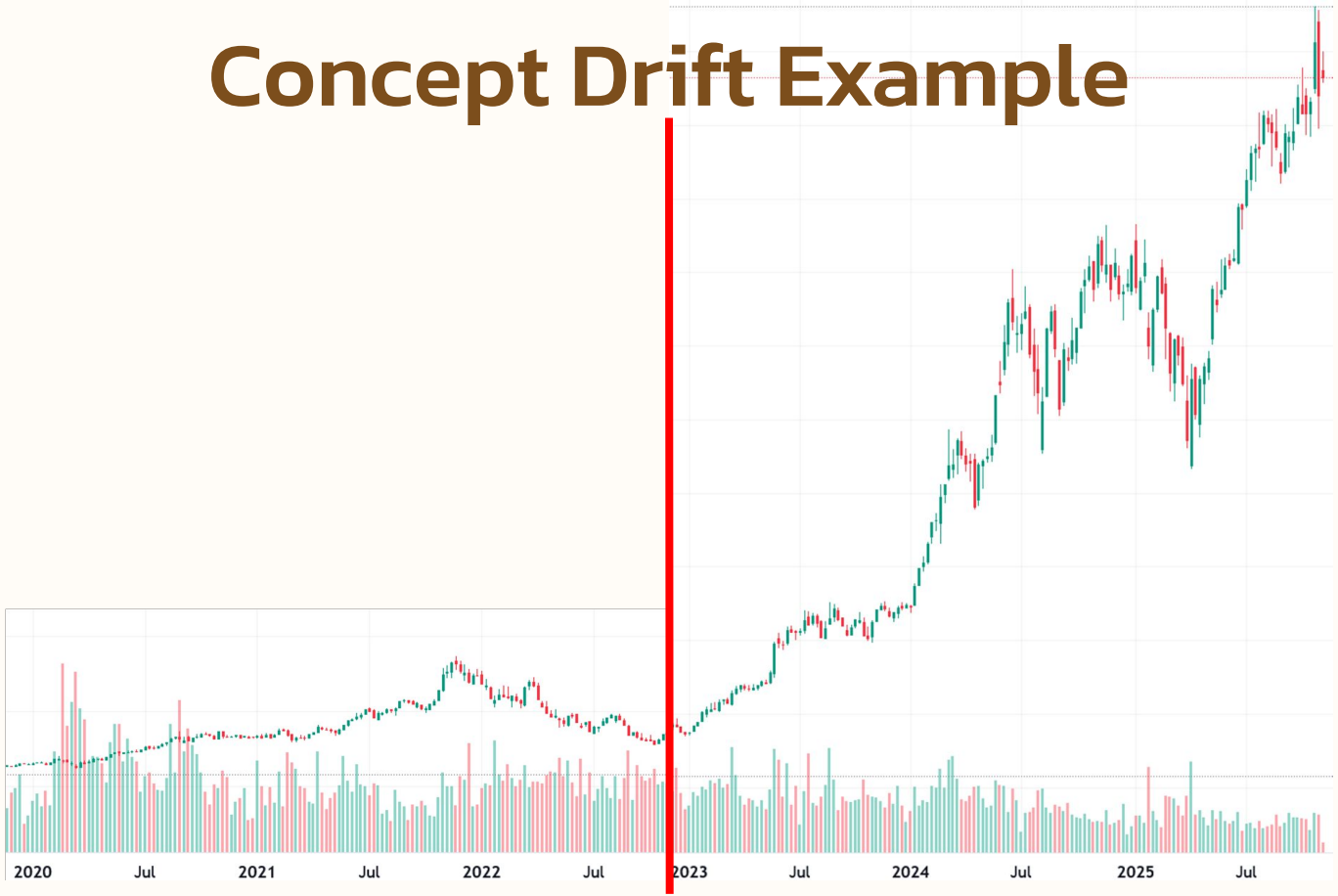
- New price dynamics
- Prices increase overall (inflation)



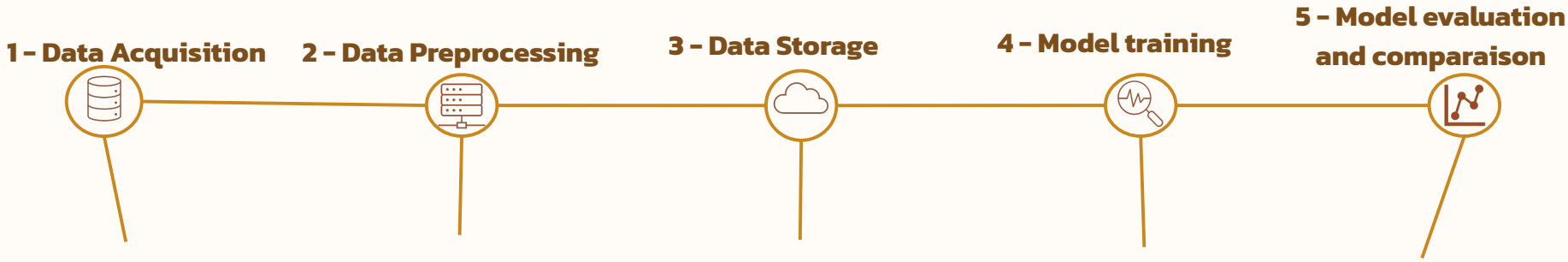
Consequences

- **Gradual performance degradation and risk of erroneous financial decisions.**

Concept Drift Example



OUR PROJECT ARCHITECTURE



1 – Data Acquisition

Collection of historical data (5 years) of 20 stocks in the NASDAQ index. Data in CSV format.

2 – Data Preprocessing

Data cleaning and transformation in a format suitable for modeling.

3 – Data Storage

Datasets and intermediate artifacts stored and versioned using DVC on an S3 bucket.

4 – Model training

Training LSTM-based forecasting model 30-day -> 5 days.

Training fully automated through the CI pipeline (DVC + GitHub Actions).

5 – Model evaluation and comparaison

Evaluation with RMSE, MAE, and R^2, plus automatic comparison across model generations via CML reports.

The "Flatline" Challenge

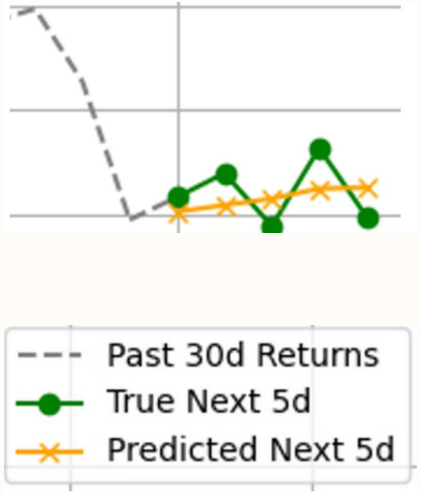
Where We Left Off

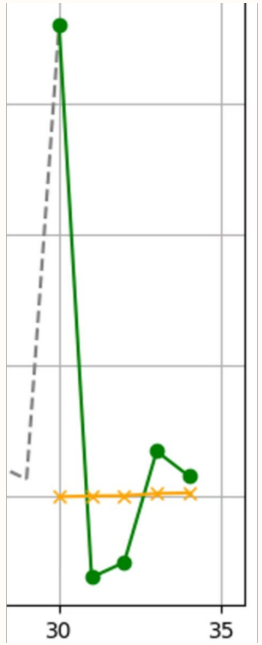
Working Pipeline

Infrastructure was in place

Failing Model

Unable to capture market volatility.





Problem: "Flatline Predictions" — merely predicting the mean.

MAE 1.87 ; RMSE: 2.87 ; R2: 0.014

Key Improvements
Since Last Presentation

1

Extended input

from 30 days to 60 days

2

Added new features

SMA, EMA, spreads, ratios, volume signals

3

Implemented drift detection

with Evidently

4

Created candidate dataset

& candidate model evaluation flow

5

Automatic performance comparison

through CML reports

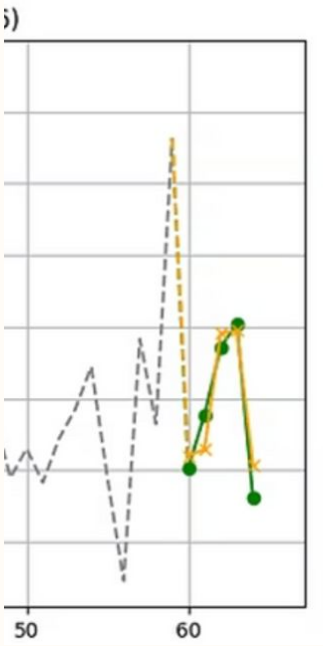
6

Deployment

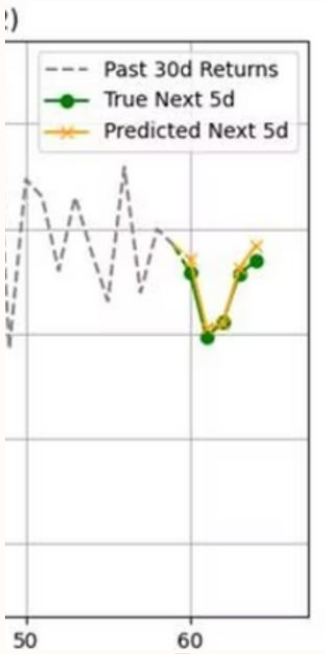
with BentoML + Docker + GKE

Final Results: Visual Performance

- RMSE: 0.10
- Predicted curve closely matches true 5-day trend



- RMSE: 0.08
- Model captures drop + recovery pattern



Global Results:

- **MAE: 0.65** → **65% better (old 1.86)**
- **RMSE: 2.87** → **70% better (old 0.84)**
- **R^2: -1.19** → **Worse (old 0.01)**

How to create the candidate dataset

1 Ingestion Strategy (The "Append" Policy)

We add the latest week's data to the existing history instead of replacing it

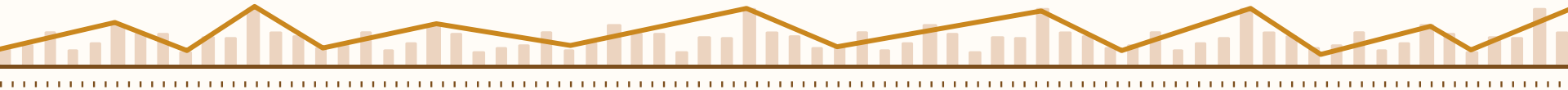
2 Dual-Dataset Structure

To enable Drift Detection, we maintain two distinct versions:

Reference Data : The fixed dataset of the active model
Candidate Data : Reference + New Week's data.

3 3. Comparison Logic

Check: Evidently compares Reference vs. Candidate distributions
Result: We detect drift before any training happens.



How We Measure Data Drift

We measure data drift in two ways:

Evently Package :

- Compare **reference** vs **candidate dataset**
- Drift detected if **Wasserstein distance > 0.5**

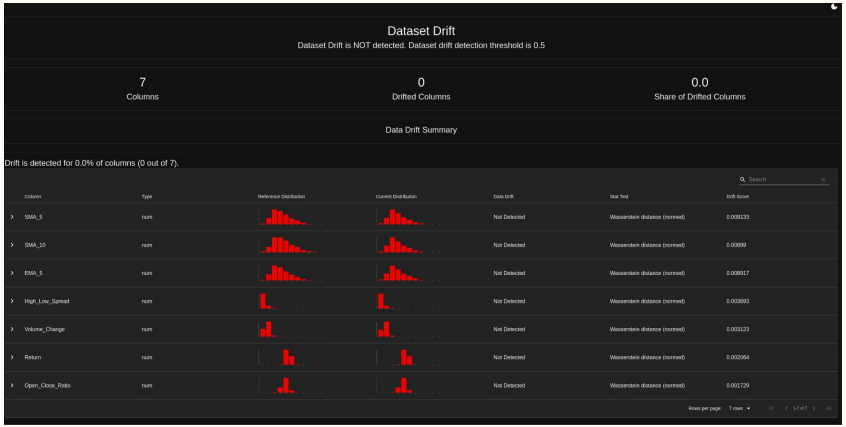
Model Performance:

- Compare **baseline model** vs **candidate model**
- Performance improvement indicates data drift

Real Data Drift Results

Observation

- **Scope:** Analyzed 7 features (SMA, EMA, Volatility, etc.) on new data
- **Result:** 0% Drift (0/7 columns)
- **Scores:** Negligible (e.g., SMA_5 ~ 0.009$, far below 0.5)



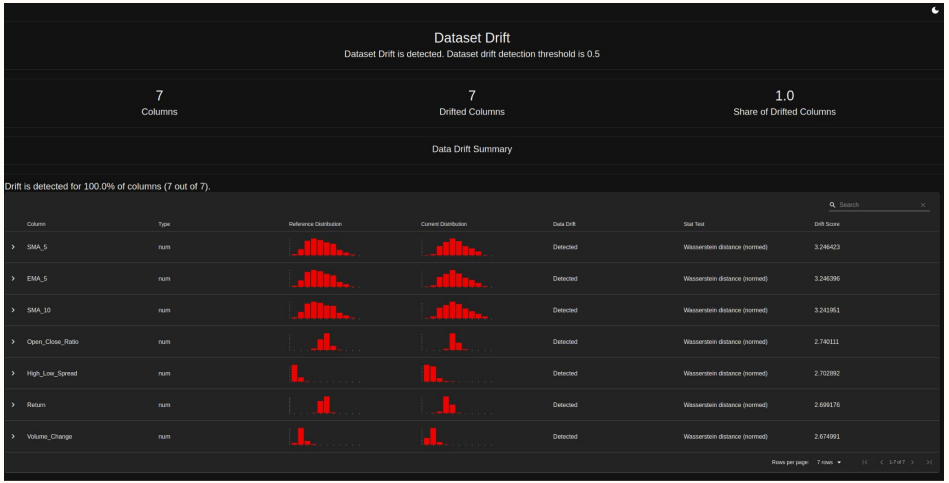
Conclusion

- Data distribution remains stable over the last month.
- Current model remains valid without immediate retraining.

Artificial Drift Validation

Goal : Since no real drift occurred, we needed to validate the alert system.

Injection Method : Modified candidate features in prepare.py: $X_{new} = X_{old} \times 2.0 + 5.0$



- **Drift Detected :** 100% (7/7 columns)
- **Scores:** High impact (> 2.0)
- **Verdict:** pipeline correctly flags significant statistical shifts

Deployment Strategy

Objective: Ensure updates improve the model, not just change it.



Method: Challenger vs. Champion

- We evaluate two models in parallel on the latest data:
  - **Baseline**: Current production model.
  - **Candidate**: New model trained on fresh data.

The "Go/No-Go" Rule

- **Deploy Candidate ⇔ Candidate RMSE < Baseline RMSE.**
  - **Result:** We only deploy when performance actually increases.

Strategy Pivot:

Candidate Model Approach

- **New dataset added**
  stored in DVC
- **Candidate model trained automatically**
- **PR comment compares metrics & plots**
- **Model deployed if accepted**

Training time per run: <2 minutes
Inference time: ~20 ms

Advantages: Scalable, Reproducible, Easy drift inspection, Avoids unnecessary retraining

Alternative Approaches

1

"Sliding Window" Dataset

Keep a fixed history (e.g., 5 years) by dropping old data

↓

Removes obsolete market behaviors & speeds up training

2

Real-Time Observability

Stream metrics via Prometheus/Grafana

↓

Instant alerts for crashes instead of weekly reports

3

Macro-Economic Context

Add VIX, Inflation, and Interest Rates features

↓

Helps the model understand why prices move, not just how

Limitations & Next Steps



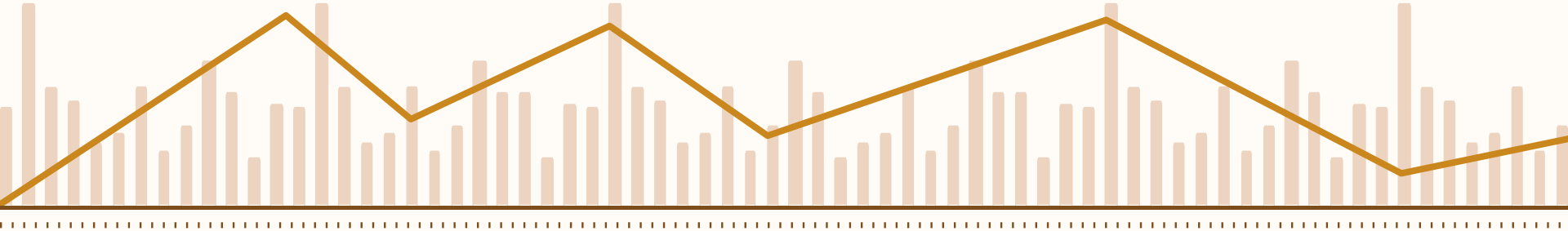
Current Limitations

- ➔ Very low R^2 → limited explanatory power
- ➔ Poor performance during high volatility
- ➔ Minimal drift observed (short time window)
- ➔ No macroeconomic features integrated



Next Steps

- ➔ Inject synthetic drift to validate detection and retraining
- ➔ Add macro variables (VIX, inflation, interest rates)
- ➔ Test a classification approach Target ≥ 70% directional accuracy



Thanks!

Do you have any questions?